

## MediKiosk — SIH Internal Hackathon Q&A

### 1. MediKiosk exactly kya hai?

Answer:

“MediKiosk ek AI-powered patient intake and triage kiosk hai jo government hospitals ke OPD mein doctor ke paas jaane se pehle patient ka initial clinical intake automate karta hai.

Patient ABHA ID se start karta hai, phir ARIA naam ka AI assistant symptoms ke baare mein questions poochta hai, patient apne reports upload/scan kar sakta hai, AI us information ko structured SOAP note mein convert karta hai, triage priority assign karta hai aur appropriate department suggest karta hai.

Uske baad doctor AI-generated information ko review karke patient ko approve karta hai.”

Simple words mein:

Patient → AI Intake → OCR → AI Summary → Triage → Department → Doctor Review → Patient

Ye complete flow project document mein defined hai.

---

### 2. Aapne ye problem hi kyun choose ki?

Answer:

“Humne ye problem isliye choose ki because government hospitals mein patient load bahut high hota hai aur OPD ka initial process repetitive aur time-consuming hai.

Doctor ka valuable time basic history lene mein chala jata hai. Saath hi wrong department routing, paper reports ko manually read karna aur first-come-first-served queue ke wajah se urgent patients identify karna difficult hota hai.

MediKiosk doctor ko replace nahi karta; instead, doctor ke consultation se pehle repetitive information-gathering ka workload reduce karta hai.”

Document mein four major bottlenecks identify kiye gaye hain: history-taking, wrong department routing, paper reports aur lack of intelligent triage.

---

3. Is problem ka real-world impact kya hai?

Answer:

“Impact mainly three levels par hai:

Patient ke liye — waiting aur confusion reduce ho sakta hai.

Doctor ke liye — consultation se pehle structured patient history available hoti hai.

Hospital ke liye — patients ko department aur priority ke basis par organize karna easier ho jata hai.”

Document ke according doctors basic history lene mein approximately 3–5 minutes per patient spend kar sakte hain, aur wrong routing se patient ko unnecessary waiting face karni pad sakti hai.

---

4. Aapka solution existing OPD system se different kaise hai?

Answer:

“Traditional OPD mein patient registration ke baad generally queue mein wait karta hai aur doctor consultation ke time history collect karta hai.

MediKiosk mein history-taking consultation se pehle AI-assisted hoti hai.

Iske saath humne four things combine ki hain:

1. Conversational AI intake

2. Medical document OCR

3. AI-based triage and department routing

4. Real-time patient-doctor synchronization

Plus multilingual voice interaction aur AYUSH-specific intake bhi included hai.”

Ye differentiators project ke comparison section mein explicitly listed hain.

---

5. Aapke solution mein “AI” exactly kaha use ho raha hai?

Answer:

“AI sirf chatbot ke form mein nahi hai. Multiple stages par use ho raha hai.

First, ARIA patient se clinical questions poochta hai.

Second, Gemini Vision uploaded reports ko read karke relevant information extract karta hai.

Third, LLM complete conversation aur OCR data ko process karke SOAP note generate karta hai.

Fourth, system triage priority aur department routing suggest karta hai.

So AI is being used as an intake, information extraction and decision-support layer.”

Project mein Groq primary LLM aur Gemini fallback/vision model ke roop mein use kiya gaya hai.

---

6. ARIA kya hai?

Answer:

“ARIA hamara AI nurse/clinical intake assistant hai.

ARIA ka purpose diagnosis dena nahi, balki patient se structured history collect karna hai.

Ye SOCRATES framework use karta hai — Site, Onset, Character, Radiation, Associations, Timing, Exacerbating/Relieving factors aur Severity.

Example: agar patient chest pain bolta hai, system automatically relevant cardiac questions par branch kar sakta hai.”

Ye branching behaviour project ke prompt architecture mein defined hai.

---

7. SOCRATES kya hai aur aapne ise kyun use kiya?

Answer:

“SOCRATES ek structured clinical history-taking framework hai.

Instead of AI randomly questions pooche, humne questions ko clinically meaningful dimensions mein organize kiya hai.

For example chest pain mein location, onset, nature, radiation, associated symptoms aur severity jaise factors important hote hain.

Isse conversation more structured aur consistent hoti hai.”

---

8. Kya MediKiosk patient ka diagnosis karta hai?

Very important answer:

“Final diagnosis nahi.

MediKiosk ka role primarily intake, summarization, triage aur decision support hai.

AI differential possibilities aur suggested investigations generate kar sakta hai, lekin final clinical decision doctor ke paas rehta hai.

Isi reason se humne doctor approval layer rakhi hai.”

Project architecture mein doctor ko SOAP/report analysis review karne ke baad patient approve karna hota hai.

---

9. Agar AI hallucinate kare to?

Answer:

“Humne hallucination ko completely eliminate claim nahi kiya hai. Instead, prototype mein multiple safeguards rakhe hain.

Department ke liye VALID\_DEPTS whitelist hai. Unknown department ko General Medicine mein fallback kiya jata hai.

SOAP generation structured JSON format mein hoti hai.

Confidence flags indicate karte hain ki kaunse fields incomplete ya low-confidence hain.

Aur sabse important — doctor final review karta hai before approval.”

Security/reliability section mein ye safeguards documented hain.

---

10. Emergency patient ko AI normal queue mein daal de to?

Answer:

“Emergency red flags ke liye separate interruption mechanism hai.

ARIA ke response mein [EMERGENCY\_FLAG] detect hone par interview stop ho sakta hai aur frontend emergency alert/siren overlay show karta hai.

So emergency case ko normal conversation continue karne ke bajay immediately flag kiya jata hai.

However, prototype stage par this is an AI-assisted safety mechanism, not a replacement for clinical emergency assessment.”

Project mein emergency red-flag detection aur instant alert explicitly included hai.

---

11. Department routing kaise hoti hai?

Answer:

“AI patient ke symptoms aur collected clinical information ko analyze karke appropriate department suggest karta hai.

For example, patient ke symptoms ke basis par Cardiology, General Medicine etc. route ho sakta hai.

Humne department validation bhi rakhi hai, taaki LLM koi arbitrary department name generate na kar sake.”

Summary pipeline mein department validation ke liye whitelist use ki gayi hai.

---

12. Triage priority kaise decide hoti hai?

Answer:

“Prototype mein four priority levels hain:

Emergency → High → Medium → Low.

High priority mein severe symptoms ya rapid worsening jaise conditions consider ki ja sakti hain.

AI summary mein confidence flags bhi generate hote hain, so doctor ko sirf priority nahi, balki underlying information quality bhi dikhti hai.”

Document ke prompt architecture mein priority logic explicitly defined hai.

---

13. SOAP note kya hai?

Answer:

“SOAP ek standard clinical documentation structure hai:

S — Subjective: patient ne kya symptoms bataye.

O — Objective: jo objective information actually available hai.

A — Assessment: possible clinical interpretations/differentials.

P — Plan: suggested next investigations/actions.

MediKiosk conversation aur OCR data ko structured SOAP format mein convert karta hai so doctor ko raw conversation read nahi karni padti.”

Project ka data model SOAP ko S, O, A, P structure mein define karta hai.

---

14. OCR ki zarurat kyun hai?

Answer:

“Government hospitals mein patients physical reports, prescriptions aur lab documents lekar aate hain.

Agar doctor ko har report manually read karni pade, time lagta hai.

Isliye patient report upload/scan karta hai, Gemini Vision relevant text aur medical information extract karta hai, aur extracted information conversation context mein inject ho jati hai.

Isse final summary mein patient ki current conversation ke saath previous report information bhi consider ho sakti hai.”

OCR pipeline ka complete flow document mein described hai.

---

15. Aapne APIs ka use kyun kiya? Khud AI model kyun nahi banaya?

🔥 Very likely SIH question.

Answer:

“Hackathon prototype ka objective end-to-end solution demonstrate karna hai, foundation model train karna nahi.

Agar hum khud LLM train karte, to large medical dataset, GPU infrastructure, training time, validation aur deployment complexity bahut increase ho jati.

Instead, APIs allow us to integrate proven AI capabilities quickly and focus our development effort on the actual innovation — patient workflow, clinical intake, OCR integration, triage pipeline, queue management, doctor review and real-time communication.

Production phase mein models ko domain-specific Indian medical data par fine-tune ya replace kiya ja sakta hai.”

Project roadmap mein future mein Indian medical corpus par LLM fine-tuning explicitly proposed hai.

---

16. “API use karna cheating nahi hai?”

Answer:

“No. APIs are an implementation method.

Humne sirf API call karke chatbot nahi banaya. APIs ko ek larger architecture ke components ke roop mein integrate kiya hai.

For example:

Frontend → FastAPI backend → AI API → structured response → triage → queue → doctor dashboard → SSE → patient.

Our innovation is in the workflow and integration layer.”

---

17. Groq aur Gemini dono kyun use kiye?

Answer:

“Dono ka role different hai.

Groq ko primary LLM inference ke liye use kiya hai because of its low-latency inference.

Gemini ko fallback aur multimodal/OCR processing ke liye use kiya hai.

Agar primary provider fail ya rate-limit ho, system fallback provider try karta hai.

So this is a multi-model fallback architecture.”

Project specifically Groq primary aur Gemini fallback/vision pipeline define karta hai.

---

18. Agar Groq API down ho gayi to?

Answer:

“Chat aur summary pipeline mein Gemini fallback rakha gaya hai.

So system single AI provider par completely dependent nahi hai.

However, we would describe this as improved resilience rather than claiming absolute 100% uptime.”

Ye distinction judge ke saamne important hai. Document multi-model fallback architecture describe karta hai.

---

19. FastAPI kyun choose kiya?

Answer:

“FastAPI Python-based hai aur hamare AI/ML stack ke saath naturally fit hota hai.

Isme asynchronous request handling, automatic OpenAPI documentation aur SSE support milta hai.

Hamare use case mein multiple APIs hain — chat, document scanning, summary generation, approval aur real-time stream — so FastAPI suitable tha.”

---

20. SSE kya hai aur WebSocket kyun nahi?

Answer:

“SSE — Server-Sent Events — server se browser ko real-time updates push karne ka mechanism hai.

Hamare case mein communication mostly one-way hai:

Doctor approves → server → patient screen.

Patient ko continuously server ko messages send nahi karne hain.

Isliye SSE simpler tha. Browser mein EventSource native support karta hai aur separate WebSocket library ki requirement nahi hai.”

---

21. Real-time patient notification kaise hota hai?

Answer:

“Patient ke token ke liye SSE connection open rehta hai.

Doctor jab dashboard se Approve Patient click karta hai, token status Pending se Approved hota hai.

SSE stream updated status patient ke browser ko push karta hai.

Patient ko refresh karne ki zarurat nahi padti.”

---

22. Database ki jagah JSON file kyun use ki?

🔥 Feasibility question

Answer:

“Because this is an internal hackathon prototype.

Humein quickly demonstrate karna tha ki complete workflow work karta hai. Isliye lightweight session\_data.json use kiya.

Production deployment ke liye ye scalable solution nahi hai. Roadmap mein PostgreSQL + Redis migration planned hai.

So we deliberately chose simplicity for prototype and scalability for the production roadmap.”

---

23. Kya ye solution technically feasible hai?

🔥 Must-prepare answer

Answer:

“Yes, as a phased deployment.

Prototype level par required components already demonstrated hain: FastAPI backend, AI APIs, OCR, browser voice APIs, SSE, structured JSON storage and web-based kiosk interface.

Production deployment ke liye next steps honge:

ABDM production integration, secure authentication, PostgreSQL/Redis, proper doctor authentication, kiosk hardware, monitoring, clinical validation, data privacy/security compliance.

So core architecture feasible hai; productionization ke liye additional engineering and regulatory work required hoga.”

Current prototype limitations and roadmap exactly in areas ko identify karte hain.

---

24. Real hospital mein MediKiosk kaise deploy hoga?

Answer:

“Hospital OPD ke registration/waiting area mein kiosk install kiya ja sakta hai.

Patient:

ABHA authentication → consent → AI interview → report scan → summary → triage → token.

Doctor ke cabin/terminal par separate dashboard hoga.

Hospital network ke through patient status aur doctor approval synchronize honge.

Future hardware integration mein camera-based document scanning, fingerprint-based ABHA authentication aur IoT vitals devices add kiye ja sakte hain.”

---

25. ABHA integration real hai ya fake?

⚠ Be honest.

Answer:

“Current hackathon prototype mein ABHA/ABDM integration mocked hai.

Humne actual production ABDM credentials claim nahi kiye hain.

Prototype mein humne ABHA data model aur workflow simulate kiya hai.

Production version mein official ABDM/FHIR integration aur required NHA credentials ke through actual integration ki jayegi.”

This limitation is explicitly acknowledged in the document.

---

26. To phir ABHA use karne ka benefit kya hai?

Answer:

“ABHA-based identity ka objective patient ko unique health identity ke saath connect karna hai.

Prototype mein iska purpose demonstrate karna hai ki patient ki identity aur medical history workflow mein kaise integrate hogi.

Production implementation mein actual ABDM ecosystem ke through authorized health records access/push kiya ja sakta hai.”

---

27. Patient ki privacy ka kya?

Answer:

“Medical data highly sensitive hai, so privacy is a major production concern.

Prototype mein API keys .env mein rakhe gaye hain aur .gitignore mein excluded hain. File uploads restricted hain — supported formats aur 10 MB limit defined hai.

Production version mein encrypted storage, proper authentication, authorization, audit logs, secure ABDM integration and appropriate healthcare data protection controls required honge.”

---

28. Kya patient ko English aani zaroori hai?

Answer:

“No.

MediKiosk multilingual voice interaction support karta hai.

Current prototype mein Hindi, English, Tamil aur Bengali support hai.

Web Speech API ka SpeechRecognition voice input ke liye aur SpeechSynthesis text-to-speech ke liye use hota hai.”

---

29. Voice support ke liye separate AI model kyun nahi banaya?

Answer:

“Prototype mein browser-native Web Speech API use kiya because it is lightweight, zero-cost and easy to deploy on kiosk hardware.

Isse voice input ko text mein convert karke existing AI pipeline mein bhej sakte hain.

Future deployment mein dedicated speech recognition/TTS services add ki ja sakti hain if better accuracy or broader language support is required.”

---

30. AYUSH integration kyun add kiya?

Answer:

“India mein healthcare ecosystem sirf allopathic consultation tak limited nahi hai.

Isliye humne AYUSH workflow ko separate mode ke roop mein design kiya.

Trigger words detect hone par system Dashavidha Pariksha-based intake mode mein switch kar sakta hai aur patient ko Ayurveda/AYUSH department route karta hai.”

---

31. MediKiosk ki uniqueness ek line mein kya hai?

🔥 Best answer:

“Hamari uniqueness ek single AI chatbot mein nahi, balki complete pre-consultation workflow mein hai — multilingual conversational intake + medical document OCR + clinical SOAP summarization + intelligent triage + department routing + real-time queue synchronization + doctor-in-the-loop approval.”

---

32. Aapka solution “just another AI chatbot” kyun nahi hai?

Answer:

“Because chatbot is only one component.

A chatbot normally conversation provide karta hai.

MediKiosk conversation ke output ko actionable hospital workflow mein convert karta hai.

Patient conversation → structured clinical data → SOAP → priority → department → queue → doctor review → approval → patient notification.

Yahi workflow integration hamara major differentiator hai.”

---

33. Doctor AI-generated report par blindly trust karega?

Answer:

“No.

Architecture intentionally doctor-in-the-loop hai.

Doctor ko SOAP, OCR reports, priority aur confidence information review karni hoti hai before approving the patient.

AI is an assistant, not the final authority.”

---

34. Confidence flags kya hain?

Answer:

“AI har clinical information ko equally reliable assume nahi karta.

For example patient pain severity clearly nahi bata pa raha hai, to system us field ko medium/low confidence ke saath flag kar sakta hai.

Doctor ko quickly pata chal jata hai ki kis information ko verify karna important hai.”

Project mein confidence flags specifically missing/low-confidence data identify karne ke liye defined hain.

---

35. Agar patient kuch important information hide kare to?

Answer:

“AI system patient ke hidden information ko magically know nahi kar sakta.

Isliye MediKiosk ko diagnostic replacement ke roop mein position nahi kar rahe.

System collected information ko organize karta hai aur missing fields ko confidence flags/uncollected fields ke through indicate karta hai.

Final verification doctor karega.”

This answer is particularly important because it demonstrates realistic understanding of AI limitations.

---

36. Agar patient internet ke bina ho to?

Answer:

“Current prototype cloud AI APIs par dependent hai, so full AI functionality ke liye network connectivity required hai.

Production deployment mein hospital intranet architecture, local caching aur potentially edge/local models consider kiye ja sakte hain.

This is an area for future engineering rather than something the current prototype claims to solve.”

---

37. Kya ye scalable hai?

Answer:

“Current prototype is not production-scale yet because JSON file storage has limitations.

But architecture is designed so individual components can be replaced.

For example:

JSON → PostgreSQL/Redis

Mock ABDM → Real ABDM/FHIR

Browser OCR → dedicated medical document pipeline

Prototype server → cloud/containerized deployment

So the architecture has a defined scalability path.”

---

38. Government hospital ke liye cost kaise control karoge?

Answer:

“Prototype mein humne lightweight and low-cost technologies choose ki hain.

FastAPI, Vanilla JS, Tailwind CDN, browser-native speech APIs aur API-based AI services se infrastructure complexity reduce hoti hai.

Production mein cost optimize karne ke liye hybrid architecture use kiya ja sakta hai — routine tasks ke liye smaller/local models aur complex tasks ke liye cloud models.”

The current stack explicitly emphasizes lightweight frontend and browser-native capabilities.

---

39. Aapne Vanilla JS kyun use kiya? React kyun nahi?

Answer:

“Hackathon prototype ke liye speed aur simplicity priority thi.

Vanilla HTML, CSS and JavaScript directly browser mein run karte hain, build tooling ka overhead kam hai aur kiosk hardware par easily deploy kiya ja sakta hai.

Production mein React ya another frontend framework adopt karna possible hai, but prototype ke scope mein Vanilla JS sufficient tha.”

40. Agar judge kahe: “Aapne bahut saare APIs use kiye hain, actual innovation kya hai?”

🔥 Strong answer:

“ APIs are the building blocks, not the complete solution.

For example, Google/Groq humein AI inference capability dete hain, but they don't provide our complete OPD workflow.

Our implementation combines those capabilities into a domain-specific pipeline:

clinical interview → OCR → context injection → SOAP generation → triage → department routing → queue management → doctor approval → real-time patient notification.

So our innovation is in system architecture, workflow integration and the problem-specific user experience.”

41. Why not simply use ChatGPT/Gemini directly for the patient?

Answer:

“General-purpose AI chatbot patient ko conversation de sakta hai, but hospital workflow integration automatically provide nahi karta.

MediKiosk mein AI output directly structured hospital actions se connected hai:

Department routing, priority, token, doctor dashboard, SOAP, OCR reports, approval, real-time notification.

So MediKiosk is not just an AI conversation layer; it is an OPD workflow system.”

42. What happens after AI generates the summary?

Answer:

“AI structured token data generate karta hai.

Usmein token ID, ABHA ID, department, priority, queue number, SOAP, reports, confidence flags and status store hote hain.

Initially status Pending hota hai.

Doctor review karke approve karta hai, then status Approved ho jata hai and patient ko SSE ke through notification milti hai.”

43. Aapka complete technical architecture explain karo.

🔥 60-second technical answer:

“Frontend side par HTML, CSS and Vanilla JavaScript based kiosk interface hai.

Patient voice/text se ARIA ke saath interact karta hai aur documents upload karta hai.

Frontend FastAPI backend ke different endpoints ko call karta hai.

Backend mein /api/chat conversational AI handle karta hai.

/api/scan\_document Gemini Vision ke through OCR karta hai.  
/api/generate\_summary complete transcript aur OCR context ko LLM ko dekar SOAP, department and priority generate karta hai.  
Data currently JSON-based session storage mein persist hota hai.  
Doctor dashboard queue ko priority ke basis par display karta hai.  
Doctor approval ke baad /api/approve token update karta hai aur /api/stream/{token\_id} SSE ke through patient ko real-time notification deta hai.”

44. APIs kaun-kaun si hain?

“Core backend APIs hain:  
/api/chat — AI conversation  
/api/scan\_document — document OCR  
/api/generate\_summary — SOAP + triage + routing  
/api/approve — doctor approval  
/api/stream/{token\_id} — real-time SSE updates.”

45. Prototype mein APIs use karne ka main reason kya tha?

Best concise answer:  
“Because SIH prototype mein humein feasibility demonstrate karni thi, not spend the entire hackathon training foundation models.  
APIs allowed us to rapidly integrate AI, OCR and speech capabilities while our team focused on the actual MediKiosk workflow and hospital-use case.  
Once the workflow is validated, individual APIs can be replaced with self-hosted or domain-specific models.”

46. What are the current limitations?

Answer:  
“Currently:  
ABDM integration mocked hai.  
Session data JSON file mein stored hai.  
Doctor panel authentication prototype mein implemented nahi hai.  
OCR Gemini Vision par dependent hai.  
Browser TTS support browser-dependent hai.  
Full production security, scalability and clinical validation abhi future scope mein hain.”

47. Future mein kya improve karoge?

Answer:  
“Next phase mein:

PostgreSQL + Redis, real ABDM/FHIR integration, kiosk camera and fingerprint integration, IoT vitals like BP and SpO<sub>2</sub>, Indian medical corpus based fine-tuning, better NLP intent detection, ICD-aware clinical processing, and additional Indian language support.  
Ye changes prototype ko production-ready direction mein le jayenge.”

48. Is project ka biggest technical challenge kya tha?

Good team answer:

“Biggest challenge sirf AI response generate karna nahi tha.  
Challenge tha AI output ko deterministic hospital workflow mein convert karna.  
For example, free-form AI response ko structured SOAP JSON mein convert karna, department whitelist ke against validate karna, priority assign karna, token create karna, doctor dashboard se synchronize karna and finally patient ko real-time notify karna.  
So integration and reliability were major technical challenges.”

49. Is project ka biggest innovation kya hai?

Answer:

“Hum single feature ko innovation nahi bolenge.

The main innovation is the integration of multiple technologies into one pre-consultation OPD workflow.

AI conversational intake + OCR + structured SOAP + triage + department routing + multilingual voice + AYUSH + real-time doctor approval ek single workflow mein connected hain.”

50. If you had only one sentence to explain MediKiosk?

 MEMORIZE THIS:

“MediKiosk is an AI-powered pre-consultation OPD system that converts a patient's conversation and medical documents into a structured clinical summary, triage priority and department-ready queue before the patient reaches the doctor.”